

# Maven + Git/GitHub + Docker

## Start-to-End Lab Workflow

A practical exam workflow: what to do, what command to use, and what to check. Use the patterns rather than memorizing every character.

### 1. START: Get or create the Maven project

If a project is already provided, enter its folder. If it is in Git, clone it first.

```
git clone <repository-URL>
cd <project-folder>
dir
# verify pom.xml exists
```

Maven project basics: source code normally goes in `src/main/java`; build output goes in `target/`.

#### Create/check the Maven POM

```
<groupId>com.example</groupId>
<artifactId>my-app</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>
```

For a WAR/Tomcat exercise, make sure packaging is **war**. Keep artifactId free of spaces.

#### Common Maven dependency shape

```
<dependency>
  <groupId>...</groupId>
  <artifactId>...</artifactId>
  <version>...</version>
  <scope>provided</scope>
</dependency>
```

Only add dependencies that actually exist and match the application. Avoid duplicate dependencies and misspelled artifactIds.

#### Maven Compiler Plugin (when Java version must be set)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

#### Build/check Maven

```
mvn clean
mvn compile
mvn test
mvn clean package
# WAR/JAR appears in target/
```

Typical lifecycle idea: clean → compile → test → package. Use the command requested by the exercise.

### 2. GIT: Initialize and connect the repository

```
git init
git status
git remote -v
git remote add origin git@github.com:<username>/<repository>.git
git remote -v
```

SSH setup, if required: generate an SSH key, add the public key to GitHub, then use the SSH remote URL.

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# add ~/.ssh/id_ed25519.pub to GitHub SSH keys
```

## Create .gitignore

```
.env
node_modules/
target/
```

Then check status so ignored files are not staged.

## 3. GIT: Everyday add → commit → push workflow

```
git status
git add .
git status
git commit -m "meaningful message"
git push -u origin main
```

For another branch, replace main with the branch name.

## Useful inspection

```
git log --oneline
git status
git remote -v
```

## Fix an incorrect commit message

```
git commit --amend -m "Correct commit message"
git push --force-with-lease # only when history rewrite is required/allowed
```

## 4. GIT: Branches and feature work

```
git switch -c feature/login
# edit files
git status
git add .
git commit -m "Implement login"
git push -u origin feature/login
```

## Switch branches with unfinished work

```
git stash
git switch <other-branch>
# work/check something
git switch feature/login
git stash pop
```

## Unstage a file without deleting its edits

```
git restore --staged app.js
```

## Merge a feature branch

```
git switch main
git merge feature/signup
# if conflict occurs: edit conflicted file(s)
git add <resolved-file>
git commit -m "Resolve merge conflict"
```

## Safely undo an already-pushed commit

```
git revert <commit-id>
git push
```

revert creates a new commit; it is preferred for safely undoing changes already shared.

## 5. PATCH FILE WORKFLOW

```
git apply <filename.patch>
git diff
mvn clean package
git status
git add .
git commit -m "Apply patch and fix bug"
git push
```

If git apply reports a failure, inspect the changes/conflicts before committing.

## 6. DOCKER: Create the Dockerfile

For the Maven WAR + Tomcat exercise, use a multi-stage build: Maven builds the WAR; Tomcat runs it.

```
FROM maven:3.9-eclipse-temurin-17

WORKDIR /app
COPY . .

RUN mvn clean package

FROM tomcat:10.1-jdk17

COPY --from=0 /app/target/*.war /usr/local/tomcat/webapps/

EXPOSE 8080
CMD ["catalina.sh","run"]
```

Memory: **Maven** → **Copy** → **Build** → **Tomcat** → **WAR** → **Port** → **Run**.

## 7. DOCKER: Build and verify your image

```
docker build -t my-maven-app:1.0 .
docker images
```

If the build fails at RUN mvn clean package, run mvn clean package locally and fix the POM/dependencies first.

## 8. DOCKER: Pull and run Tomcat

```
docker pull tomcat:10.1-jdk17
docker run -d -p 7070:8080 tomcat:10.1-jdk17
docker ps
```

Port pattern: **host:container**. Here 7070 on your PC maps to Tomcat's 8080 inside the container.

### If host port 7070 is already allocated

```
docker ps
# stop the container using the port if appropriate
docker stop <container-id>

# or use another host port
docker run -d -p 7071:8080 tomcat:10.1-jdk17
docker ps
```

## 9. DOCKER: Deploy and verify the WAR

```
docker cp target/LMS.war <container-id>:/usr/local/tomcat/webapps/
docker ps
```

Then open the mapped port in a browser, for example <http://localhost:7070> or the WAR context such as /LMS.

## 10. DOCKER HUB: Tag, login, push

```
docker images
docker tag my-maven-app:1.0 <username>/my-maven-app:1.0
docker login
docker push <username>/my-maven-app:1.0
```

Then verify the repository/image on Docker Hub.

## 11. FULL EXAM-DAY FLOW

1. git clone <URL>
2. cd <project-folder>
3. verify pom.xml
4. update pom.xml if required (WAR, dependencies, Java version)
5. mvn clean package
6. create Dockerfile
7. docker build -t my-maven-app:1.0 .
8. docker images
9. docker pull tomcat:10.1-jdk17

```

10. docker run -d -p 7070:8080 tomcat:10.1-jdk17
11. docker ps
12. docker cp target/LMS.war <container-id>:/usr/local/tomcat/webapps/
13. open localhost:7070
14. if port conflict: use 7071:8080
15. docker tag my-maven-app:1.0 <username>/my-maven-app:1.0
16. docker login
17. docker push <username>/my-maven-app:1.0

```

## 12. COMMAND MEMORY MAP

| Goal                   | Command pattern                             |
|------------------------|---------------------------------------------|
| Get project            | git clone <URL>                             |
| Check Git              | git status / git log --oneline              |
| Stage                  | git add .                                   |
| Save change            | git commit -m "message"                     |
| Upload Git branch      | git push -u origin <branch>                 |
| Branch                 | git switch -c <branch>                      |
| Temporarily save edits | git stash / git stash pop                   |
| Merge                  | git merge <branch>                          |
| Undo shared commit     | git revert <id>                             |
| Build Maven            | mvn clean package                           |
| Build Docker image     | docker build -t <name>:<tag> .              |
| See images             | docker images                               |
| Get image              | docker pull <image>                         |
| Start container        | docker run -d -p <host>:<container> <image> |
| See containers         | docker ps                                   |
| Copy file              | docker cp <file> <container>:<path>         |
| Docker Hub name        | docker tag <image> <user>/<repo>:<tag>      |
| Docker Hub login       | docker login                                |
| Upload image           | docker push <user>/<repo>:<tag>             |

Important: This sheet is a practical workflow based on the Maven/Git/GitHub/Docker tasks covered in the supplied lab sample. Exact versions, repository names, WAR names, ports, and branches may change with a question; substitute the values given in the question.